Programming Project / C++ – Parking Management System –

Student 1: Md Redoanur Rahman Rafi
Student 2: Omar Lekhal

I. Task Description

This project is a command-line parking payment application written in C++. It allows users to register parking payments, view payment history, and see available commands. No keyboard input is used – all input is passed through command-line arguments.

Student 1 (Md. Rafi) is responsible for the application logic and file I/O:

- Implementing `HistoryRepository` for reading and writing `history.txt`
- Implementing `CommandHandler` to parse and run CLI commands
- Implementing the top-level `Application` class
- Wiring everything together and handling error output

Student 2 (Omar Lekhal) is responsible for the data and validation side:

- Implementing the `ParkingRecord` class and its file serialization
- Implementing `ParkingZone` with the pricing rules per zone
- Implementing `RomanianPlate` for license plate format checking
- Writing the validator classes (`IValidator`, `ZoneValidator`, `HoursValidator`, `CarPlateValidator`

II. Data Structures Used by the Team

The following classes are used in the project:

RomanianPlate – utility class with static methods to validate and normalize Romanian license plates (e.g. TM99XYZ, B12ABC, B123ABC).

ParkingZone – utility class that maps zone names to hourly rates. Zones are red (1.5 RON/h), yellow (1.0 RON/h), and green (2.0 RON/h).

ParkingRecord – represents one parking payment entry:

```
string car      // license plate
string street   // street name
string zone     // parking zone (Red / Yellow / Green)
int hours       // number of hours paid for
float price     // total cost in RON
```

IValidator – abstract base class that defines a common interface for all validators:

```
virtual bool validate(const string& input) const = 0;
virtual string expectedFormat() const = 0;
```

ZoneValidator, **HoursValidator**, **CarPlateValidator** – concrete classes that inherit from `IValidator` and each validate one type of input.

HistoryRepository – reads and writes the history file. Loads records into a `vector<ParkingRecord>` and appends new ones to disk.

CommandHandler – parses the CLI arguments and calls the right logic. It holds a reference to `HistoryRepository` (association).

Application – top-level class that owns a `HistoryRepository` and a `CommandHandler` via `unique_ptr` (composition). Calls `handler->execute(argc, argv)` in `run()`.

Class relationships used:

- **Inheritance**: `ZoneValidator`, `HoursValidator`, and `CarPlateValidator` all extend `IValidator`
- **Composition**: `Application` owns `HistoryRepository` and `CommandHandler`
- **Association**: `CommandHandler` holds a reference to `HistoryRepository`

III. File Structure

history.txt

Stores all parking payment records. Each line is one record, with fields separated by `|`:

```
<car_plate>|<street>|<zone>|<hours>|<price>
<car_plate>|<street>|<zone>|<hours>|<price>
...
```

Example:

```
TM99XYZ|Vasile Alecsandri Street|Red|3|4.50
B12ABC|Liberty Square|Green|2|4.00
B123ABC|Piata Unirii|Yellow|2|2.00
```

A new line is appended every time the `pay` command runs successfully. The `history` command reads the whole file and prints all records. If the file does not exist yet, `history` prints nothing and `pay` creates the file automatically.

IV. Interacting with the Executable

The application compiles to a single executable. All commands are passed as arguments.

Show help

```
./parking.exe help
```

Prints all available commands with usage hints.

View payment history

```
./parking.exe history
```

Reads `history.txt` and prints every record. Zone name is shown in yellow and price in green.

Register a parking payment

```
./parking.exe pay <car_plate> <zone> <hours> <street>

<car_plate>  Romanian license plate (e.g. TM99XYZ, B12ABC,
   B123ABC)
<zone>       red, yellow, or green (case-insensitive)
<hours>      number of hours as an integer between 1 and 24
<street>     name of the street where the car is parked
```

The command validates all arguments first. If something is wrong it prints what was invalid and what the correct format should be. On success it calculates the price (zone rate x hours), saves the record to `history.txt`, and prints a confirmation.

Example:

```
./parking.exe pay TM99XYZ red 3 "Vasile Alecsandri Street"
```

The program exits with code 0 on success and code 1 on any error.